

Odin + raylib + OpenXR

A PC VR renderer in about 900 lines — how it works, what each file does, and why the tricky parts are the way they are.

What this project is

This is a working VR application written in Odin. It draws a scene in stereo to a headset, tracks head and controller motion, and reads buttons, triggers, and thumbsticks from Meta Touch controllers. It runs on Windows as a PCVR app, driven either through SteamVR or through Meta's Link runtime.

Two libraries do the work. **raylib** provides the OpenGL context and all the drawing — meshes, models, textures, shaders. **OpenXR** provides everything specific to VR — where the head is, what the lens distortion requires of the projection matrix, where the controllers are, and the textures the compositor will display.

Neither library knows about the other. The job of this codebase is to introduce them, and one function does most of that work.

The central idea

raylib's `RenderTexture2D` is not a magic handle. It is a plain struct holding OpenGL object ids:

```
RenderTexture2D :: struct {  
    id: u32, // framebuffer object  
    texture: Texture2D, // color attachment  
    depth: Texture2D, // depth attachment  
}
```

Nothing requires those ids to have come from raylib. OpenXR hands out swapchain images as raw OpenGL texture ids, so we create a framebuffer ourselves, attach OpenXR's texture as the color target, and fill in the struct by hand. From that point on raylib believes it is drawing into an ordinary render texture, and every normal raylib draw call lands in the headset's swapchain.

Why this matters

This is what makes the whole approach viable. Without it you would be writing a renderer from scratch against Vulkan or raw OpenGL before seeing a single triangle in the headset. With it, `DrawModeLEx`, materials, and shaders all work unchanged.

What happens in one frame

Understanding this sequence explains most of the code. Every VR frame runs the same eight steps, and OpenXR is strict about their order.

```
xrWaitFrame <- blocks here; this is what paces the app to the
headset's refresh rate, not vsync
xrBeginFrame
xrLocateViews <- ask: where is each eye, right now, and what is
its field of view?
for each eye:
  xrAcquireSwapchainImage <- runtime gives us an image index
  xrWaitSwapchainImage <- wait until it is safe to draw into
  render_eye(...) <- raylib draws the scene
  xrReleaseSwapchainImage <- hand it back
xrEndFrame <- submit both eyes as one projection layer
```

render.odin — xr_frame

The loop in `main.odin` calls this once per iteration, then separately draws the desktop mirror window. Those two must not be interleaved: `EndDrawing` swaps the desktop buffers, and calling it between the two eyes corrupts the submission.

Why BeginMode3D cannot be used

raylib's `Camera3D` describes a camera with a single fovy value, which implies a symmetric view frustum — the same angle left as right, up as down.

Headset lenses are not symmetric. Each eye sees further toward the outside of the face than toward the nose, so OpenXR reports four separate angles per eye. A projection matrix built from those has off-center terms that fovy simply cannot express. Feeding a symmetric matrix to a headset produces an image that looks almost right and feels subtly wrong.

So `render_eye` inlines the body of `BeginMode3D` and substitutes OpenXR's matrices. This is the same thing raylib's own `BeginVrStereoMode` does internally.

```
rlgl.MatrixMode(rlgl.PROJECTION)
rlgl.PushMatrix()
rlgl.LoadIdentity()
rlgl.MultMatrixf(raw_data(proj_f[:])) // from xr_projection()

rlgl.MatrixMode(rlgl.MODELVIEW)
rlgl.LoadIdentity()
rlgl.MultMatrixf(raw_data(view_f[:])) // from xr_view_matrix()

rlgl.EnableDepthTest()
draw_scene() // ordinary raylib calls
```

render.odin — the substituted matrix setup

The files

Odin allows one package per directory, so every file is package `main` and they all see each other with no imports between them. The split is organisational only.

File	Lines	What it holds
<code>scene.odin</code>	93	Your world. <code>draw_scene</code> , <code>update</code> , <code>pointer</code> . The file you edit day to day.
<code>main.odin</code>	44	Window creation, initialisation order, the frame loop.
<code>render.odin</code>	170	<code>render_eye</code> , <code>xr_frame</code> , and the desktop mirror window.
<code>xr_context.odin</code>	34	The XR struct and the single <code>xr_ctx</code> global.
<code>xr_init.odin</code>	118	Instance, system, session, reference space, and teardown.
<code>xr_swapchain.odin</code>	156	Swapchain creation and the <code>RenderTarget2D</code> wrapping trick.
<code>xr_input.odin</code>	395	Action set, Meta Touch bindings, per-frame controller state.
<code>xr_events.odin</code>	55	The session state machine.
<code>xr_math.odin</code>	94	Projection and view matrices, column-major packing.
<code>gl.odin</code>	71	Raw GL/WGL entry points, constants, discrete-GPU exports.

Roughly speaking: `scene.odin` is yours, `render.odin` is the bridge, and everything prefixed `xr_` is plumbing you should rarely need to open again.

File by file

scene.odin

Three procedures. `draw_scene` draws the world with ordinary raylib calls. `update` runs game logic once per frame. `pointer` draws the controllers and their aim rays.

What works and what does not inside draw_scene:

Works	Does not work
<code>DrawCube</code> , <code>DrawSphere</code> , <code>DrawLine3D</code> , <code>DrawGrid</code>	<code>BeginMode3D</code> / <code>EndMode3D</code> — overwrites the XR matrices
<code>DrawModel</code> , <code>DrawModelEx</code> , <code>DrawMesh</code>	<code>BeginDrawing</code> / <code>EndDrawing</code> — owned by <code>render.odin</code>
<code>LoadTexture</code> , materials, <code>BeginShaderMode</code>	<code>DrawText</code> , <code>DrawRectangle</code> — screen-space, lands nowhere useful

Units are real now

1.0 is one metre and the origin sits on your physical floor. A cube at {0, 1.2, -2} is chest height, two metres away. Scene dimensions carried over from a desktop project will be enormous.

xr_swapchain.odin

Creates one swapchain per eye and pre-builds a RenderTexture2D around every image in it. The runtime cycles through those images; `xrAcquireSwapchainImage` says which index to use each frame.

```
wrap_swapchain_image :: proc(gl_tex: u32, w, h: i32) -> rl.RenderTexture2D {
    target: rl.RenderTexture2D

    target.id = rlgl.LoadFramebuffer()
    rlgl.EnableFramebuffer(target.id)

    // Color: OpenXR's texture. Never unload this one.
    target.texture.id = gl_tex
    target.texture.width = w
    target.texture.height = h
    target.texture.format = .UNCOMPRESSED_R8G8B8A8

    // Depth: ours. OpenXR supplies none without the depth-layer extension.
    target.depth.id = rlgl.LoadTextureDepth(w, h, true)

    rlgl.FramebufferAttach(target.id, target.texture.id,
        ATTACH_COLOR_CHANNEL0, ATTACH_TEXTURE2D, 0)
    rlgl.FramebufferAttach(target.id, target.depth.id,
        ATTACH_DEPTH, ATTACH_RENDERBUFFER, 0)
    rlgl.FramebufferComplete(target.id)
    rlgl.DisableFramebuffer()

    return target
}
```

xr_swapchain.odin — the load-bearing function

The split ownership matters at shutdown. The framebuffer and the depth buffer are ours to delete; the color texture belongs to OpenXR. Calling `UnloadRenderTexture` would delete all three and corrupt the runtime's swapchain.

`RENDER_SCALE` in this file divides the runtime's recommended eye resolution. That recommendation already includes whatever supersampling multiplier SteamVR is configured with, which can be extreme — over 3000×3000 per eye is common.

xr_input.odin

The largest file, and the one with the least obvious design, because OpenXR's input model is indirect by design.

You never read a button. You declare an abstract *action* — "trigger", "jump" — then *suggest* which physical inputs it maps to on each type of controller. The runtime performs the actual binding. This is why the same code runs on Touch, Index, and WMR hardware, and why users can rebind your app in SteamVR's settings without your cooperation.

The setup order is fixed and one-shot:

1. `xrCreateActionSet`
2. `xrCreateAction` <- ALL of them; none can be added later
3. `xrSuggestInteractionProfileBindings` <- once per controller profile
4. `xrCreateActionSpace` <- for pose actions only
5. `xrAttachSessionActionSets` <- permanent for the session

then every frame:
`xrSyncActions`
`xrGetActionStateBoolean / Float / Vector2f` per action, per hand
`xrLocateSpace` for poses

Reading input from your code:

```
right := hand(.Right)

right.trigger // f32, 0..1
right.trigger_pressed // true only on the frame it crossed the threshold
right.stick // [2]f32, -1..1, y is +up
right.a // A held (X on the left controller)
right.a_pressed // just pressed this frame
right.aim // xr.Posef – pointing ray
right.grip // xr.Posef – physical hand position

rumble(.Right, 0.05, 0, 0.6) // duration s, frequency Hz, amplitude 0..1
```

scene.odin — typical usage

Field	Type	Notes
active	bool	controller is currently tracked
aim / grip	xr.Posef	pointing ray / physical hand position
trigger / squeeze	f32	0..1 analog
stick	[2]f32	-1..1 per axis, y is +up
a / b	bool	A and B on the right; X and Y on the left
stick_click / menu	bool	menu exists on the left controller only

Field	Type	Notes
*_pressed / *_released	bool	true only on the frame of the change

Digital edges come free: OpenXR reports `changedSinceLastSync` alongside the current state, so `pressed = current && changed` with no manual previous-state tracking. Analog inputs have no such flag, so `trigger_pressed` is a threshold crossing compared against a stored previous value.

Two quirks of Meta hardware

The button naming asymmetry is the controllers' own: X and Y live on the left, A and B on the right, so a single action maps to X on one hand and A on the other. And the right controller's system button is reserved by the runtime and cannot be bound by applications, so menu only exists on the left.

Adding a button is four edits, all in this file: an `Action` field, a `make_action` call, a binding path, and a read in `xr_sync_input`. Because attachment is permanent, an action you forget cannot be added without restarting the session.

xr_events.odin

The session state machine, and the single most common reason a first OpenXR application shows a black headset with no error messages anywhere.

Creating a session does not start it. The runtime posts state changes as events, and nothing renders until it reaches READY and you respond by calling `xrBeginSession`. STOPPING must be handled too, or the application hangs when the user takes the headset off.

```
IDLE -> READY -> SYNCHRONIZED -> VISIBLE -> FOCUSED
^ |
+-- STOPPING <-----+

READY call xrBeginSession here, or nothing will ever display
SYNCHRONIZED frames are being paced, but shouldRender is false
VISIBLE your frames are displayed; a system overlay has focus
FOCUSED your frames are displayed and you receive input
```

The distinction between VISIBLE and FOCUSED is worth knowing when debugging: at VISIBLE rendering works but `xrSyncActions` returns `SESSION_NOT_FOCUSED` and controller input stops updating. That is normal — it happens whenever the system menu is open.

xr_math.odin

Small file, high density of things that break subtly rather than loudly.

View matrix composition order

OpenXR gives a camera-to-world transform; a view matrix is its inverse. The composition is `translate * rotate`. Reversed, the head position gets rotated by the head orientation, and the world appears to slide toward and away from you as you turn your head — a symptom that reads as tracking failure but is pure matrix algebra.

Column-major packing

Odin's `rl.Matrix` is the transpose of raylib's C `Matrix`. Rather than guess whether the binding compensates when handing a matrix to a C helper, `matrix_to_gl` writes the sixteen floats explicitly in the order OpenGL expects.

Asymmetric projection

`xr_projection` builds a frustum from OpenXR's four separate FOV angles. The off-center terms are what `Camera3D` cannot represent.

gl.odin

Everything raylib does not expose. Three groups:

- **WGL context handoff.** OpenXR needs the HDC and HGLRC of the context it will share textures with. `wglGetCurrentDC` and `wglGetCurrentContext` return whatever is current on the calling thread, so calling them on the main thread after `InitWindow` retrieves raylib's context without

digging into its internal GLFW handles.

- **sRGB control.** `glEnable(GL_FRAMEBUFFER_SRGB)`, exposed as a runtime toggle.
- **Discrete GPU exports.** Two exported symbols that opt a laptop out of its integrated GPU. See the debugging section — this one costs hours.

Debugging: symptoms and causes

Every entry here was hit during development. VR failures tend to be silent — the runtime accepts everything and simply shows nothing — so the mapping from symptom to cause is worth keeping.

Symptom	Cause and fix
Compositor sits at "waiting" forever; app reports endFrame=SUCCESS	Rendering on the integrated GPU. OpenGL textures cannot be shared across adapters, so the compositor receives handles it cannot read. The exported NvOptimusEnablement / AmdPowerXpressRequestHighPerformance symbols in gl.odin opt out; otherwise set the exe to High performance in Windows graphics settings. main prints the GL renderer at startup — check it first.
Black headset, no errors, mirror window fine	Session state machine never reached READY, or xrBeginSession was never called. Watch the printed state transitions.
Frames submitted but nothing displays; state stuck at SYNCHRONIZED	shouldRender is false because another scene application holds focus. Quit SteamVR Home and any other VR app.
World slides toward and away as you turn your head	View matrix composed as rotate * translate. Must be translate * rotate.
Image looks almost right but feels wrong	Symmetric projection matrix. Camera3D cannot express the asymmetric per-eye FOV; use xr_projection.
Scene much too dark, or washed out	The runtime offered no linear RGBA8 swapchain, so gamma is applied a different number of times than raylib expects. Press F to toggle GL_FRAMEBUFFER_SRGB and keep whichever looks correct.
Terrible framerate	The recommended eye resolution includes SteamVR's supersampling multiplier and can exceed 3000 pixels square. Lower RENDER_SCALE.
Crash or corruption on exit	UnloadRenderTexture deletes OpenXR's color texture. Unload only the framebuffer and your own depth buffer.
Controller input freezes but rendering continues	Session is VISIBLE rather than FOCUSED — a system overlay has focus and xrSyncActions returns SESSION_NOT_FOCUSED. Normal behaviour.

Bring-up procedure

When something breaks, reduce before investigating. Comment draw_scene() and pointer() out of render_eye so only ClearBackground runs.

- A solid color in both eyes means the entire OpenXR side is healthy and the problem is in your scene code.
- Black means the problem is upstream — session state, GPU selection, or swapchain setup. The console output narrows it immediately.

Extending it

Add geometry

Anything that draws in 3D works inside `draw_scene`, unchanged from desktop raylib. Note that `DrawCube` has no texture parameter — textured geometry needs a model:

```
// once, at startup
tex := rl.LoadTexture("crate.png")
model := rl.LoadModelFromMesh(rl.GenMeshCube(1, 1, 1))
model.materials[0].maps[i32(rl.MaterialMapIndex.ALBEDO)].texture = tex

// in draw_scene
rl.DrawModelEx(model, position, axis, angle, scale, rl.WHITE)
```

The tint argument multiplies into the texture rather than replacing it, so pass `WHITE` unless you want it colored.

Add a controller button

Four edits, all in `xr_input.odin`:

```
// 1. a field on Input
thumbrest_action: xr.Action,

// 2. create it in xr_init_actions, before AttachSessionActionSets
make_action("thumbrest", "Thumbrest Touch", .BOOLEAN_INPUT,
&in_.thumbrest_action)

// 3. a binding in xr_suggest_touch_bindings
{in_.thumbrest_action, path("/user/hand/left/input/thumbrest/touch")},
{in_.thumbrest_action, path("/user/hand/right/input/thumbrest/touch")},

// 4. read it in xr_sync_input
t_now, t_changed := get_bool(in_.thumbrest_action, i)
h.thumbrest = t_now
```

Add UI

Screen-space calls like `DrawText` have no meaning in a headset. Two approaches work: draw geometry in world space and raycast against it with `hand(.Right).aim`, or render a conventional 2D interface into a separate `RenderTarget2D` and map that texture onto a quad floating in front of the user.

Add lighting

The default raylib shader is unlit — everything renders as flat albedo. Real shading means loading the lighting shader from raylib's shaders example and assigning it to your materials. That works normally here; it is orthogonal to the VR plumbing.

Known limits

- **Windows only** as written. Linux needs the Xlib or Wayland graphics binding struct in place of the Win32 one; nothing else changes.
- **PCVR only.** Odin has no Android target, so standalone Quest builds are not possible. The headset must be tethered via Link, Air Link, or SteamVR.
- **OpenGL only.** The Vulkan path would be considerably faster and is what commercial engines use, but it means writing a Vulkan renderer first.
- **Community bindings.** The OpenXR bindings are generated from the Khronos registry by a third party, not part of Odin's vendor collection. Names may drift between versions.

Setup: clone the `openxr_odin` bindings and copy the `openxr /` folder next to these sources, then `odin run . -out:editor.exe` with the headset connected.